FIAP · DOMAIN DRIVEN DESIGN USING JAVA · SPRINT 4

Pointway API

API RESTful em Spring Boot que converte pontos digitais da plataforma SoulUp em créditos reais de transporte público — documentação final do projeto Java (Desafio 2: Utilização de Pontos para Transporte Público).

NOME DA EQUIPE	INTEGRANTES	SOLUÇÃO
Equipe Pointway	**Kauã Fiuza — RM572369** **Lucas Almeida — RM572499** **Enzo Leiva — RM568877** **Matheus Segura — RM570878**	**Pointway (módulo SoulTransporte)**

Repositório GitHub: `https://github.com/SEU-USUARIO/pointway-api`
Deploy da API: `https://SEU-LINK-DO-DEPLOY`

SUMÁRIO

Índice do documento

01	**Objetivo e escopo do projeto**	Problema, propósito e limites	p. 3
02	**Funcionalidades implementadas**	O que a API entrega ao front-end	p. 4
03	**Tabela de endpoints**	URIs, verbos HTTP e códigos de status	p. 5
04	**Protótipo — telas implementadas**	Prints do front-end consumindo a API	p. 7
05	**Modelo de Entidade-Relacionamento**	Tabelas, chaves e relacionamentos	p. 8
06	**Diagrama de classes**	Camadas model, DAO, BO e resource	p. 9
07	**Procedimentos para rodar a aplicação**	Ferramentas, versões, execução e deploy	p. 10

01 · OBJETIVO E ESCOPO

Transformar engajamento digital em mobilidade real

A SoulUp identificou que grande parte dos pontos acumulados por seus usuários expira sem uso, enquanto o transporte público é uma necessidade diária — principalmente para a população de baixa renda. O **Pointway** resolve essa desconexão: o usuário converte pontos digitais em créditos de transporte, usados por meio de um voucher digital em ônibus, metrô, trem e BRT de operadoras parceiras.

O propósito do sistema é dar um destino concreto e cotidiano aos pontos SoulUp, gerando impacto ambiental mensurável (CO_2 evitado) e inclusão social, ao reduzir o custo de transporte para quem mais depende dele.

O que mudou da Sprint 3 para a Sprint 4

Na Sprint 3 o projeto era uma aplicação Java com camada de modelo, DAO e uma classe de teste. Nesta entrega final ele virou uma **API RESTful em Spring Boot**, publicada na nuvem e consumida pelo front-end em React + Vite + TypeScript. As entidades, o banco e as regras de negócio foram mantidos; foram adicionadas a camada **BO** (validações e regras), a camada **Resource** (endpoints), o **CorsFilter**, o tratamento padronizado de erros e o deploy.

Escopo desta entrega: cadastro e gestão de usuários e operadoras; crédito de pontos; conversão de pontos em vouchers com regra antifraude; validação do voucher na catraca com registro da viagem e cálculo de CO_2; painel do usuário e indicadores gerais de impacto; persistência em MySQL com CRUD completo das quatro tabelas.

Fora do escopo: integração real com os sistemas de bilhetagem das operadoras (SPTrans, Metrô, RioCard), autenticação de usuários e o aplicativo mobile. A validação na catraca é simulada pelo endpoint `POST /viagens`.

02 · FUNCIONALIDADES

O que a API entrega ao front-end

Cadastro de usuários

CRUD completo, CPF validado pelos dígitos verificadores e único no sistema, e-mail validado.

Crédito de pontos SoulUp

Endpoint para somar pontos ganhos na plataforma ao saldo do usuário, com limite por operação.

Operadoras parceiras

CRUD com filtro por cidade; 4 operadoras já são cadastradas na primeira execução.

Conversão de pontos em voucher

100 pontos = R$ 1,00, mínimo de 500 pontos, saldo verificado e debitado na emissão.

Prevenção de fraude

No máximo 3 vouchers emitidos por usuário em 24 horas.

Voucher digital

Código único (UUID), validade de 30 dias, prorrogável até 90; vencidos viram EXPIRADO automaticamente.

Validação na catraca

Uso único: registra a viagem, marca o voucher como UTILIZADO e recusa voucher vencido ou já usado.

Cálculo de CO_2 economizado

Distância × fator do modal (ônibus 0,09 · metrô 0,04 · trem 0,05 · BRT 0,07 kg/km).

Cancelamento com estorno

Voucher ainda não usado pode ser cancelado e os pontos voltam ao saldo.

Painel e impacto

Resumo por usuário (saldo, vouchers, viagens, CO_2) e indicadores gerais para a página inicial.

Erros padronizados

Toda falha volta em JSON com status HTTP e mensagem pronta para exibir na tela.

Integridade dos dados

Usuário ou operadora com vouchers vinculados não pode ser excluído.

Tabela de endpoints

URL base: `https://SEU-LINK-DO-DEPLOY` (local: `http://localhost:8080`). Corpo e respostas em JSON.

Verbo	URI	Descrição	Sucesso	Erros
Geral				
GET	`/`	Status da API (health check)	200	—
GET	`/impacto`	Indicadores gerais da plataforma	200	500
GET	`/modais`	Modais aceitos e fator de CO_2	200	—
Usuários				
GET	`/usuarios`	Lista todos os usuários	200	500
GET	`/usuarios/{id}`	Busca usuário pelo id	200	404
GET	`/usuarios/cpf/{cpf}`	Busca usuário pelo CPF	200	400, 404
GET	`/usuarios/{id}/resumo`	Painel: saldo, vouchers, viagens e CO_2	200	404
POST	`/usuarios`	Cadastra usuário	201	400, 409
PUT	`/usuarios/{id}`	Atualiza usuário	200	400, 404, 409
PATCH	`/usuarios/{id}/pontos`	Credita pontos SoulUp	200	400, 404, 422
DELETE	`/usuarios/{id}`	Remove usuário sem vouchers	204	404, 409
Operadoras				
GET	`/operadoras?cidade=`	Lista operadoras (filtro opcional)	200	500
GET	`/operadoras/{id}`	Busca operadora	200	404
POST	`/operadoras`	Cadastra operadora	201	400
PUT	`/operadoras/{id}`	Atualiza operadora	200	400, 404
DELETE	`/operadoras/{id}`	Remove operadora sem vouchers	204	404, 409

Verbo	URI	Descrição	Sucesso	Erros
Vouchers				
GET	`/vouchers?usuarioId=&status=`	Lista vouchers (filtros opcionais)	200	400, 404
GET	`/vouchers/{id}`	Busca voucher pelo id	200	404
GET	`/vouchers/codigo/{codigo}`	Busca voucher pelo código	200	404
POST	`/vouchers`	Converte pontos em voucher	201	400, 404, 422
PUT	`/vouchers/{id}`	Altera validade e/ou status	200	400, 404, 422
DELETE	`/vouchers/{id}`	Cancela e devolve os pontos	204	404, 422
Viagens				
GET	`/viagens?usuarioId=`	Lista viagens (filtro opcional)	200	404
GET	`/viagens/{id}`	Busca viagem	200	404
POST	`/viagens`	Usa o voucher na catraca e registra a viagem	201	400, 404, 422
PUT	`/viagens/{id}`	Corrige modal/distância (recalcula CO_2)	200	400, 404
DELETE	`/viagens/{id}`	Remove viagem	204	404

Significado dos códigos de status

Código	Quando acontece
200	Consulta ou atualização feita com sucesso.
201	Registro criado; o header `Location` traz a URI do novo recurso.
204	Registro removido (sem corpo na resposta).
400	Dados inválidos: CPF/e-mail errado, campo obrigatório vazio, JSON mal formado, modal inexistente.
404	Usuário, operadora, voucher, viagem ou endpoint não encontrado.
409	Conflito: CPF já cadastrado ou registro com vouchers vinculados.
422	Regra de negócio: saldo insuficiente, limite de 3 resgates em 24 h, voucher já usado ou vencido.
500	Erro interno ou falha de conexão com o banco de dados.

Exemplo — converter pontos em voucher

```
POST /vouchers
{
  "usuarioId": 1,
  "operadoraId": 1,
  "pontos": 800
}
```

```
422 Unprocessable Entity
{
  "status": 422,
  "erro": "Unprocessable Entity",
  "mensagem": "Saldo insuficiente. Saldo atual: 300 pontos.",
  "caminho": "/vouchers"
}
```

04 · PROTÓTIPO

Telas implementadas

Capturas reais do front-end (React + Vite + TypeScript, publicado na Vercel) consumindo esta API.

[Inserir print da tela inicial
— consome GET /impacto]

1. Início — mostra os indicadores gerais (vouchers emitidos, viagens e CO_2 evitado) vindos de `GET /impacto`.

[Inserir print do cadastro / perfil
— POST /usuarios e GET /usuarios/{id}/resumo]

2. Cadastro e painel — o usuário se cadastra e vê saldo, vouchers e impacto acumulado.

[Inserir print da conversão
— GET /operadoras e POST /vouchers]

3. Conversão — escolhe a operadora e a quantidade de pontos; a API devolve o voucher ou a mensagem de erro.

[Inserir print da validação
— POST /viagens]

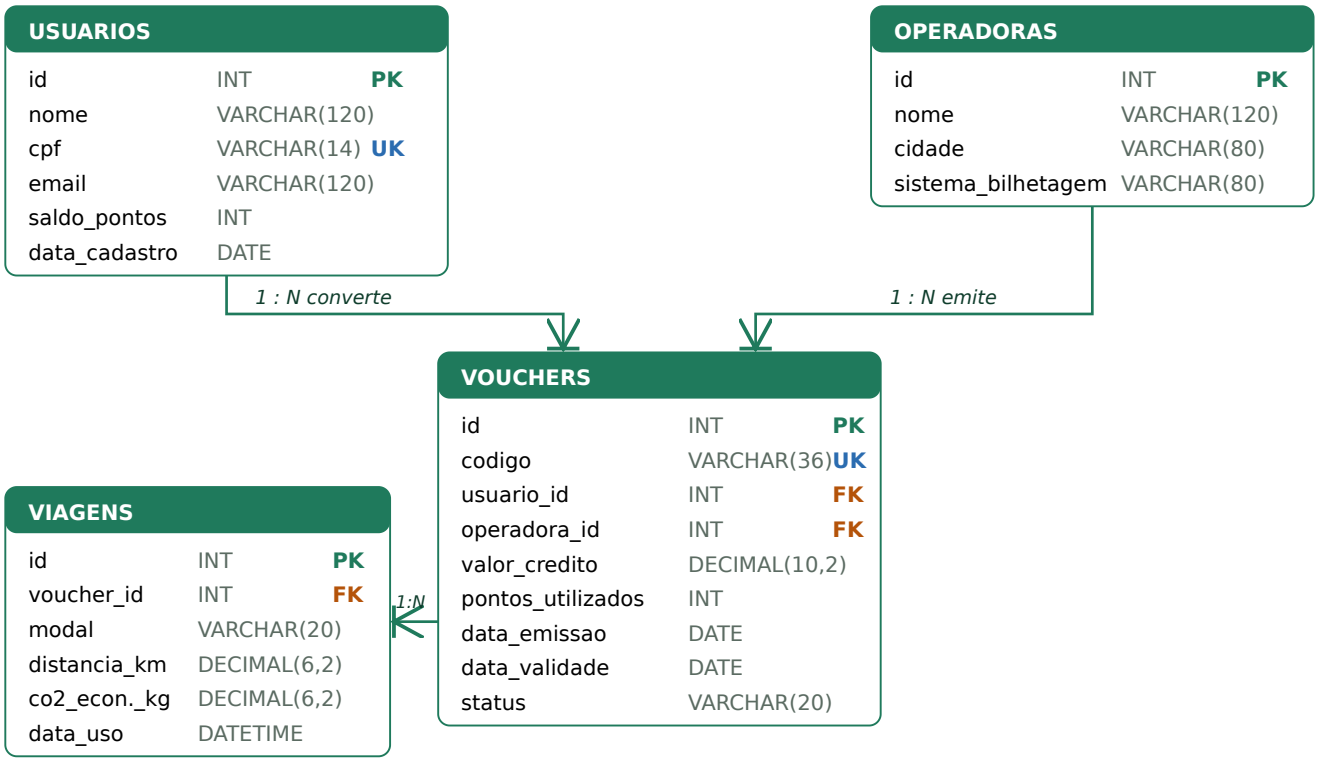
4. Validação na catraca — informa modal e distância; a viagem é registrada com o CO_2 economizado.

> Substituir os quadros acima pelos prints reais das telas do front-end antes da entrega.

05 · MODELO DE ENTIDADE-RELACIONAMENTO

Estrutura do banco de dados

O banco é o mesmo da Sprint 3 (MySQL). As tabelas são criadas automaticamente pela API ao iniciar, a partir de `schema.sql`.

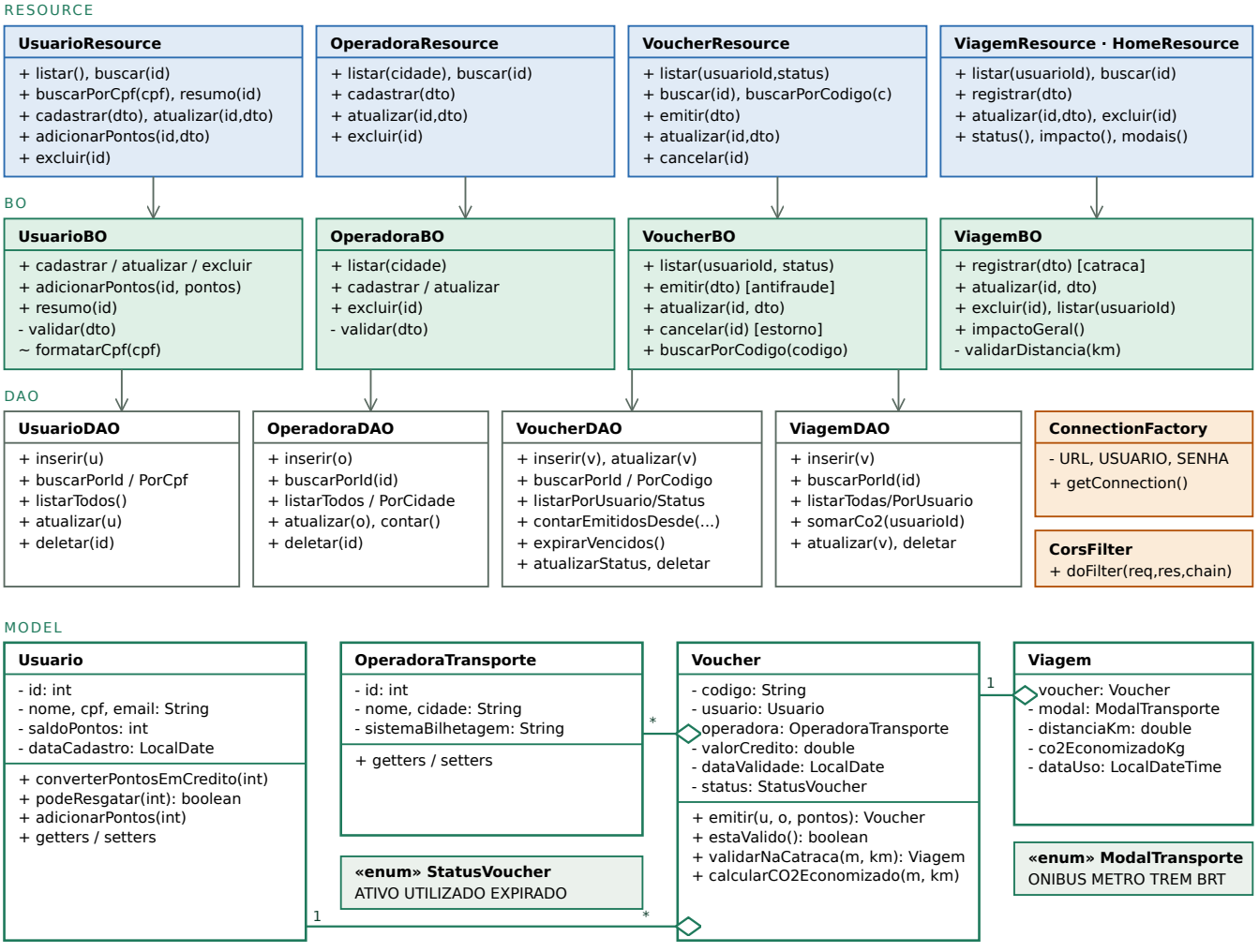


Relacionamento	Cardinalidade	Regra
USUARIOS → VOUCHERS	1 : N	Um usuário converte pontos em vários vouchers (fk_voucher_usuario).
OPERADORAS → VOUCHERS	1 : N	Cada voucher vale em uma operadora (fk_voucher_operadora).
VOUCHERS → VIAGENS	1 : N (na prática 1 : 1)	O voucher é de uso único; a regra de negócio garante uma viagem por voucher (fk_viagem_voucher).

06 · DIAGRAMA DE CLASSES

Camadas da aplicação

Fluxo de uma requisição: **Resource** (recebe o HTTP) → **BO** (valida e aplica regras) → **DAO** (SQL via **ConnectionFactory**) → MySQL. O **model** carrega as regras de domínio.



Setas cinza: dependência (injeção pelo construtor). Losango: associação. Apoio: DTOs, exceções, GlobalExceptionHandler, DatabaseInitializer.

Do zero até a API no ar

Ferramentas e versões

Item	Versão
Java JDK	17 ou superior
Spring Boot	3.5.6
Maven	3.9+
MySQL	8.x
IDE	IntelliJ IDEA, Eclipse ou VS Code (Extension Pack for Java)
Testes	Postman, Insomnia ou `requests.http`

Importar o projeto

IntelliJ: File > Open > pasta `pointway-api` (com o `pom.xml`).

Eclipse: File > Import > Maven > Existing Maven Projects.

VS Code: abrir a pasta com o Extension Pack for Java.

O Maven baixa o Spring Boot e o driver do MySQL sozinho.

Executar localmente

1. Iniciar o MySQL e criar o banco: `CREATE DATABASE pointway_db;` (ou rodar `database/pointway_schema.sql`).
2. Conferir usuário e senha do banco em `dao/ConnectionFactory.java` (padrão `root`/`root`). Também é possível usar as variáveis de ambiente `DB_URL`, `DB_USER` e `DB_PASSWORD`.
3. Executar a classe `PointwayApplication` pela IDE, ou no terminal: `mvn spring-boot:run`.
4. Abrir `http://localhost:8080/` — deve aparecer `"status": "online"`. As tabelas e 4 operadoras de exemplo são criadas na primeira execução.
5. Testar os endpoints com o arquivo `requests.http` ou pelo Postman.

Deploy na nuvem (Railway)

1. Publicar o projeto no GitHub.
2. No Railway: **New Project > Deploy from GitHub repo**. O `Dockerfile` do projeto é detectado e a API é compilada com Maven e executada com Java 17.
3. No mesmo projeto: **+ New > Database > MySQL**.
4. No serviço da API, aba **Variables**, criar `DB_URL`, `DB_USER` e `DB_PASSWORD` apontando para o MySQL do Railway (valores no README).
5. **Settings > Networking > Generate Domain**: o link gerado é a URL da API usada pelo front-end na Vercel.

```
DB_URL=jdbc:mysql://${{MySQL.MYSQLHOST}}:${{MySQL.MYSQLPORT}}/${{MySQL.MYSQLDATABASE}}?
useSSL=false&allowPublicKeyRetrieval=true
DB_USER=${{MySQL.MYSQLUSER}}
DB_PASSWORD=${{MySQL.MYSQLPASSWORD}}
```

Integração com o front-end

O `CorsFilter` libera chamadas vindas de outro domínio (a Vercel) e responde às requisições *preflight* (`OPTIONS`). No React, a URL da API fica na variável `VITE_API_URL`:

```
const resp = await fetch(`${import.meta.env.VITE_API_URL}/vouchers`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ usuarioId: 1, operadoraId: 1, pontos: 800 }),
});
if (!resp.ok) alert((await resp.json()).mensagem);
```

Boas práticas aplicadas

- Padrões DAO, Connection Factory e MVC (Resource → BO → DAO → Model).
- Injeção de dependência pelo construtor e nomes em português seguindo a convenção Java.
- `PreparedStatement` em todo SQL (evita SQL Injection) e *try-with-resources* para fechar conexões.
- Exceções próprias (`ValidacaoException`, `RegraNegocioException`, `ConflitoException`, `RecursoNaoEncontradoException`) convertidas em status HTTP pelo `GlobalExceptionHandler`.
- Status REST corretos: 201 com header `Location` na criação, 204 na remoção.